



ALAB 319.4.1:

Aggregations, Indexes, and Validation

Version 1.0, 07/31/23

[Click here to open in a separate window.](#)

Introduction

This assignment will ask you to expand the example REST API application that was explored during the lessons, adding additional features for aggregations, indexes, and validation.

Feel free to reference the lesson materials, MongoDB documentation, and other resources when creating these additional features, but think critically about how you want to approach your own solution. Sometimes, the way things are presented the first time are not the best way to do them!

Objectives

- Add additional features to an existing MongoDB + Express CRUD API.
- Refactor existing code for efficiency, organization, and/or performance.

Submission

Submit your completed lab using the **Start Assignment** button on the assignment page in Canvas.

Your submission should include:

- A link to the GitHub repository for your project.

Instructions

You will begin with the example `grades` application that was explored during the lessons.

- If you have already created a working local copy of that application, wonderful! Continue with that as the starting point for this assignment.
- Otherwise, download [the final example CodeSandbox](#) using the top-left menu.
 - Once downloaded, rename the directory to an appropriate project name.
 - Run `git init` to establish a git repository within the directory.
 - Run `npm install` to install the application dependencies.
 - Add your own environment variables to enable connection to your database.
 - Test the application by running `nodemon index.js` and navigating to `localhost:3000`.

Throughout the assignment, commit frequently to your Git repository.

Part 1: Exploring Existing Operations

Take a few minutes to explore the existing code again. Make sure you are familiar with the structure and functionality of what you will be working with. Whenever you are given somebody else's code to work on, it is important to take the appropriate time to understand it before attempting modifications and improvements.

Part 2: Adding Additional Features

Your task is to add the following additional features, and any code necessary to make them work as described. The pre-existing code should remain functional! Assume that you have clients actively using the existing code, so maintaining it is necessary for the health of the application.

Note that some of these features can be handled through the use of MongoDB Compass and/or the MongoDB Shell. Use whatever method you believe to be most effective and efficient for each task.

Create the following features, using good organizational and coding practices:

- Create a GET route at `/grades/stats`
 - Within this route, create an aggregation pipeline that returns the following information:
 - The number of learners with a weighted average (as calculated by the existing routes) higher than 70%.
 - The total number of learners.
 - The percentage of learners with an average above 70% (a ratio of the above two outputs).
- Create a GET route at `/grades/stats/:id`
 - Within this route, mimic the above aggregation pipeline, but *only* for learners within a class that has a `class_id` equal to the specified `:id`.
- Create a single-field index on `class_id`.
- Create a single-field index on `learner_id`.
- Create a compound index on `learner_id` and `class_id`, in that order, both ascending.
- Create the following validation rules on the `grades` collection:
 - Each document must have a `class_id` field, which must be an integer between 0 and 300, inclusive.
 - Each document must have a `learner_id` field, which must be an integer greater than or equal to 0.
- Change the validation action to "warn."

Part 3: Testing

Test your features! Make sure that everything works as expected.

If code does not work, leave comments for yourself explaining potential next steps, so you can revisit the application in the future and approach the problem again from a new perspective.

If code prevents the application from running, comment it out before submission.

Part 4: Completion

Upload your project to a GitHub repository, and submit it according to the submission instructions at the beginning of this document.